

Chapter 10 : Knowledge Representation

1. What is Knowledge Representation?

- Knowledge representation (KR) is a way of encoding information about the world so that an AI agent can reason with it.
- It provides a formal language to represent facts, objects, relations, and general knowledge.
- KR is the foundation for reasoning, planning, learning, and decision making.

Key Idea: A knowledge representation specifies what exists in the world and how the world works, in a form that supports inference.

2. Why Knowledge Matters in AI

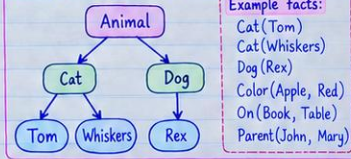
- Pure search is not enough for complex, real-world problems.
- Knowledge helps an agent:
 - Reduce the search space.
 - Make inferences about unseen situations.
 - Handle incomplete or uncertain information.
 - Generalize to new situations.
 - Act intelligently in complex environments.

Example: Knowing that "if it is raining, the ground is wet" allows an agent to infer Wet(Ground) from Rain().

3. Objects, Categories, Properties, and Relations

- Objects: individual things in the world (e.g., Tom).
- Categories (classes): groups of similar objects (e.g., Cat).
- Properties: attributes of objects (e.g., color).
- Relations: connections between objects (e.g., On, Parent).

Example: Objects and Categories



4. Ontological Commitment

- A knowledge representation language commits us to a particular view of the world — an ontology.
- It defines what kinds of objects, properties, and relations exist.
- Different applications may require different ontological commitments.

Examples of ontological commitments:

- Are there abstract objects such as numbers?
- Are events or time points part of the ontology?
- Are there mental objects such as beliefs?

5. Events, Actions, and Time

- The world is dynamic: things happen and change over time.
- We represent events and actions and their effects on the world.
- Time can be represented using time points, intervals, or situation calculus.
- Actions have preconditions and effects.

Example: A simple action

Action: Move(Agent, From, To)

Precondition: At(Agent, From)

Effect: $\neg \text{At}(\text{Agent}, \text{From}) \wedge \text{At}(\text{Agent}, \text{To})$

6. Agents and Mental Objects

- KR can represent mental attitudes such as beliefs, goals, intentions, and knowledge.
- Mental objects allow us to reason about what agents know, believe, or want.
- Useful for multi-agent systems and modeling other agents.

Examples:

- Believes(John, Rain())
- Goal(John, At(Home))
- Knows(John, Cat(Tom))
- Intends(John, Move(Home, School))



7. Facts vs. Defaults

- Some knowledge is definite (facts).
- Often we also have typical or default knowledge that holds in most cases but can have exceptions.
- KR formalisms must distinguish between strict facts and default assumptions.

Facts (always true)

Cat(Tom)
Color(Apple, Red)
Parent(John, Mary)

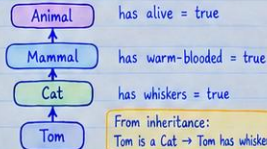
Default knowledge (usually true)

Bird(x) $\rightarrow$ Flies(x)
(true for most birds, but not for penguins)

8. The Role of Inheritance

- Inheritance lets us infer properties of objects from their categories (classes).
- If Tom is a Cat and cats typically have whiskers, we can infer that Tom has whiskers.
- Inheritance supports compact representation of general knowledge and enables default reasoning.

Inheritance Example



9. Examples from the Real World

Domain	Objects / Relations	Example Facts
Animals	Cats, Dogs, HasColor, ISA	Cat(Tom) Color(Tom, Brown)
People	Person, Parent, Friend, Age	Person(John) Parent(John, Mary)
Physical world	Room, Location, On, In, Connected	On(Book, Table) In(Key, Drawer)
Actions	Move, PickUp, Action, Causes	Move(Robot, A, B) Causes(Move, At)
Time	TimePoint, Before, After, During	Happens(Rain, t1) Before(t1, t2)

10. Key Takeaways

- KR is about representing information so that agents can reason.
- Knowledge reduces the search space and enables intelligent behavior.
- We represent objects, categories, properties, and relations.
- A KR language commits us to an ontology.
- The world includes events, actions, and time.
- We can represent mental objects such as beliefs and goals.
- Distinguishing between definite facts and default (typical) knowledge.
- Inheritance supports generalization and default reasoning.
- KR is essential for many real-world AI applications.
- The choice of representation affects what can be expressed and how efficiently we can reason.

1. Syntax and Semantics of Representation

- Syntax specifies the legal symbols and structure of sentences in a representation language.
- Semantics specifies the meaning of sentences in terms of the world.
- A knowledge representation language must have both a well-defined syntax and semantics.
- Semantics maps sentences to models (possible worlds) where the sentence is true.

Key Idea: Syntax tells us what sentences are well-formed; semantics tells us what they mean.

2. Logic as a Representation Language

- Propositional and first-order logic are formal languages for knowledge representation.
- Sentences represent facts about the world.
- Semantics is defined in terms of models and truth (model-theoretic semantics).
- Inference derives new sentences that are entailed by the knowledge base.
- Logic provides a precise, unambiguous representation with clear semantics.

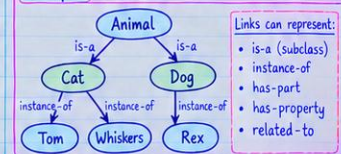
Example: In first-order logic:

- $Cat(Tom)$ (Tom is a cat)
- $\forall x Cat(x) \rightarrow Mammal(x)$
- $Mammal(Tom)$ (inferred)

3. Semantic Networks

- A graphical representation of knowledge using nodes and labeled links.
- Nodes represent objects or concepts.
- Links represent relations (e.g., is-a, instance-of, has-part, attribute).
- Supports inheritance of properties.
- Provides a simple and intuitive way to represent taxonomic knowledge.

Example: A semantic network for animals



4. Frames

- A frame is a structured representation for a stereotyped situation, object, or concept.
- Consists of a collection of slots and values.
- Slots represent attributes or relations.
- Supports default values, inheritance, and procedural attachments.
- Useful for representing complex objects with many properties.

Example: Frame for a Cat

Frame: Cat	
is-a:	Animal
has-color:	(default: Brown)
has-parts:	Head, Tail, Legs
can:	(default: Climb)
sound:	(default: Meow)
lives-in:	(default: House)

5. Slot-Filler Structures

- Slots are attributes or relations of a frame.
- Fillers are the values for the slots.
- Fillers can be constants, other frames, lists, or procedures.
- Slots can have default values, constraints, and procedural attachments (e.g., if-needed, if-added).

Example: Slot fillers in a Cat frame

Slot	Filler
is-a	Animal (frame)
name	Tom (constant)
has-color	Brown (default)
has-owner	John (constant)
has-pet	Behavior (procedure)
has-friends	[Whiskers, Rex] (list)

6. Scripts

- A script represents a stereotyped sequence of events in a familiar situation.
- Organizes knowledge about what typically happens, in what order.
- Consists of a set of scenes, each with expected roles, objects, and actions.
- Useful for representing common activities (e.g., a restaurant visit, a trip, a doctor visit).

Example: Restaurant script (simplified)



7. Production Rules

- A production rule has the form:
IF condition(s) THEN action(s)
- Conditions are patterns that match the current situation (working memory).
- Actions can add, delete, or modify knowledge, or perform external actions.
- Production systems are simple and effective for many AI applications (e.g., expert systems).

Example: Rule for diagnosis

IF Fever(x) $\wedge$ Cough(x)
THEN Diagnose(x, Flu)

8. Declarative vs. Procedural Knowledge

- Declarative knowledge: represents facts about the world (what is true).
- Procedural knowledge: represents how to do something (how to achieve a goal).
- Both are important; many KR schemes combine the two (e.g., frames with procedures, production rules).

Comparison:

Aspect	Declarative Knowledge	Procedural Knowledge
Represents	Facts, relationships	Actions, methods
Focus	What is true	How to do something
Example	$Cat(Tom)$	IF ... THEN ...
KR schemes	Logic, semantic networks, frames	Production rules, procedures
Use	Reasoning about world	Control, behavior, problem solving

9. Advantages and Limitations of KR Schemes

- Each KR scheme has its own strengths and weaknesses.
- Choice depends on the application, the type of knowledge, and the reasoning needed.

Comparison of KR Schemes:

Scheme	Advantages	Limitations
Logic	Precise semantics, powerful inference	Can be complex, hard to represent commonsense
Semantic networks	Intuitive, good for taxonomies	Limited expressiveness, inheritance issues
Frames	Structured, modular, defaults	Less formal, can be ad hoc
Scripts	Good for stereotyped situations	Not suitable for novel situations
Production rules	Simple, modular, easy to implement	Can be inefficient for large knowledge bases

10. Key Takeaways (Page 2 of 5)

- Knowledge representation languages have syntax and semantics.
- Logic provides a formal and precise representation language.
- Semantic networks represent knowledge using nodes and labeled links.
- Frames use slots and fillers to represent structured objects with defaults.
- Scripts represent stereotyped sequences of events.

- Production rules represent knowledge as condition-action rules.
- Declarative knowledge represents what is true; procedural knowledge represents how to do something.
- Different KR schemes have different advantages and limitations.
- The best choice of KR scheme depends on the application and the type of knowledge.

1. Propositional vs. First-Order Representations

Propositional Representation

- Uses propositional symbols (atomic facts).
- No internal structure (no objects, properties, or relations).
- Each sentence is a single symbol.
- Simple, easy to implement, but limited expressiveness.

Example: Raining, $\neg$ Raining, Hot, AtHome

First-Order Style Representation

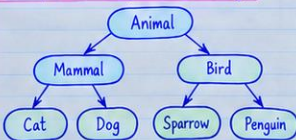
- Uses objects, properties, functions, and relations.
- Can express general statements with variables and quantifiers.
- More expressive and closer to natural language.

Example: Raining(), Person(x), At(John, Home), $\forall x$ Human(x) $\rightarrow$ Mortal(x)

3. Taxonomies and ISA Hierarchies

- A taxonomy organizes concepts into a hierarchy using ISA (is-a) relationships.
- Represents generalization and specialization.
- Allows inheritance of properties from parent to child.
- Supports efficient organization of knowledge about categories.

Example: Taxonomy (ISA Hierarchy):



ISA means:

- Cat is a Mammal
- Mammal is an Animal
- Sparrow is a Bird
- Bird is an Animal

4. Inheritance

- Children inherit properties from their parent categories.
- Inheritance allows general knowledge to apply to many specific cases.
- Properties can be inherited along multiple paths in a hierarchy.
- Supports compact representation and default reasoning.

Example: Inheritance of Properties



5. Exceptions and Default Reasoning

- Inheritance is typically monotonic, but real-world knowledge has exceptions.
- Default reasoning assumes a property holds unless there is evidence to the contrary.
- More specific information overrides more general defaults.

Example: Default and Exception

Normally birds can fly: Bird(x) $\Rightarrow$ CanFly(x) (default)
Penguins are birds: Penguin(x) $\Rightarrow$ Bird(x)
Penguins do not fly: Penguin(x) $\Rightarrow \neg$ CanFly(x) (exception)
Therefore, Tweety (a sparrow) can fly by default,
but Opus (a penguin) cannot fly due to the exception.

6. Closed-World Assumption (CWA)

- Assumes that what is not known to be true is false.
- Common in databases and many AI applications.
- Makes reasoning simpler and faster.
- Useful when knowledge is expected to be complete.

Example:

- If the KB does not contain Flies(Tom), then under CWA we conclude $\neg$ Flies(Tom).

7. Open-World Assumption (OWA)

- Does not assume that unknown facts are false.
- The truth value of an unknown fact remains undecided (could be true or false).
- Common in first-order logic and the semantic web.
- More flexible and better for incomplete knowledge.

Example:

- If the KB does not contain Flies(Tom), then under OWA we do not know whether Flies(Tom) is true or false.

8. Closed-World vs. Open-World Assumption

Aspect	Closed-World Assumption (CWA)	Open-World Assumption (OWA)
Unknown facts	Assumed false	Unknown (could be true or false)
Typical use	Databases, practical AI systems	First-order logic, semantic web
Knowledge base	Considered complete	May be incomplete
Reasoning style	Negation as failure	No negation as failure
Example conclusion	$\neg$ Flies(Tom) if not known	No conclusion about Flies(Tom)

11. Real-World Examples

Domain	Knowledge Representation	Example
Biology	Taxonomies + properties	Cat is a mammal, birds can fly (default)
Medicine	Rules with exceptions	Normally aspirin reduces fever, but may fail
Robotics	Objects, relations, actions	At(Robot, Room1), Connected(Room1, Room2)
Geography	Hierarchies and relations	Paris is a city, France is a country, In(Paris, France)
Web search	Ontologies and knowledge bases	Person(BarackObama), BornIn(BarackObama, Hawaii)

9. Knowledge Bases

- A knowledge base (KB) is a set of sentences in a KR language.
- Stores facts, rules, and general knowledge about the world.
- Can be updated with new information.
- Used by inference algorithms to derive new conclusions.

Example KB (First-Order Style)

Cat(Tom) $\forall x$ (Cat(x) $\Rightarrow$ Mammal(x))
Mammal(x) $\Rightarrow$ Animal(x)
Penguin(Opus)
Penguin(x) $\Rightarrow$ Bird(x)
Penguin(x) $\Rightarrow \neg$ Flies(x)

10. Querying a Knowledge Base

- We can ask queries to find out if a sentence follows from the KB.
- Queries may be yes/no questions, or requests for specific information.
- Inference algorithms (e.g., resolution, forward/backward chaining) are used to derive answers.

Example Queries

- Is Tom a mammal? (Yes)
- Can Opus fly? (No)
- Is Tweety an animal? (Yes)
- What are the animals in the KB?
- Find all birds that cannot fly.

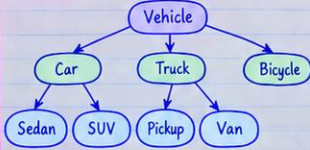
12. Key Summary

1. Propositional and first-order representations differ in expressiveness.
2. Symbols require an interpretation to have meaning.
3. Taxonomies organize concepts using ISA hierarchies.
4. Inheritance allows properties to be shared by categories.
5. Exceptions and default reasoning handle special but defeasible knowledge.
6. Closed-world assumption treats unknown facts as false.
7. Open-world assumption leaves unknown facts undecided.
8. A knowledge base stores sentences about the world.
9. We can query a knowledge base to derive new information.
10. These ideas are widely used in real-world AI applications.

1. Ontology and Categories

- An ontology specifies the kinds of things that exist in the domain and how they are organized into categories (classes).
- Categories help us group objects with similar properties and support inheritance.
- Typical categories: objects, events, actions, properties, relations, and time points.

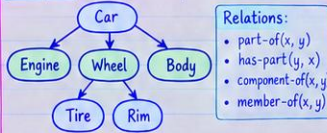
Example: A simple ontology of vehicles.



2. Part-Whole Relations

- Part-whole (mereology) relates an object to its parts and to the whole.
- Common relations: part-of, has-part, component-of, member-of (for collections).
- Part-whole knowledge is useful for modeling physical objects and hierarchies.

Example: A car and its parts.



- Relations:**
- part-of(x, y)
 - has-part(y, x)
 - component-of(x, y)
 - member-of(x, y)

If part-of(Engine, Car) is true, we can infer has-part(Car, Engine).

3. Causal Knowledge

- Causal knowledge represents cause-effect relationships between events or actions.
- Helps predict the consequences of actions.
- Useful for explanation, planning, and reasoning about the effects of changes.

Example: A simple causal chain.



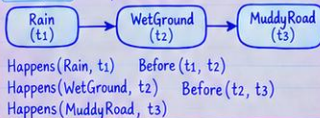
Properties:

- Causes can be actions or events.
- Effects may have further causes.
- Causal knowledge often includes conditions (context, preconditions).
- Supports prediction, planning and explanation (why something happened).

4. Representing Time

- Many domains require reasoning about when things happen.
- Time can be represented using time points, intervals, or a discrete sequence of steps.
- Common relations: Before, After, During, At(t), and Happens(event, t).
- Temporal knowledge helps reason about change, duration, and ordering of events.

Example: Temporal relations.



Happens(Rain, t1) Before(t1, t2)
Happens(WetGround, t2) Before(t2, t3)
Happens(MuddyRoad, t3)

5. Representing Actions

- Actions are represented as relations with parameters, e.g., Move(Agent, From, To).
- We represent what actions do, rather than how to do them (declarative representation).
- Actions change the state of the world.
- Actions can be instantaneous or durative.

Example: Move action.

Action:	Move(Agent, From, To)
Parameters:	Agent, From, To
Intuition:	Agent moves from one location to another.

6. Preconditions and Effects

- An action is applicable only if its preconditions are true in the current state.
- The action results in a set of effects that add or remove facts from the state.
- Effects can be conditional and may include both positive and negative effects.

Example: Pickup action.

Action: Pickup(Agent, Object)	
Preconditions:	Effects:
• At(Agent, Location)	• Holding(Agent, Object)
• At(Object, Location)	• ¬At(Object, Location)
• Clear(Agent)	• ¬Clear(Agent)

7. Situation Calculus (Overview)

- The situation calculus is a formal language for representing dynamically changing worlds.
- A situation is a history (sequence of actions).
- Fluents are properties that can change over situations.
- Actions are modeled using successor state axioms (successor state axioms).

Key Notions:

- Situation: $S_0, S_1, S_2, \dots$
- Fluent: $On(x, y, s)$, $Holding(x, s)$
- Action: $Do(a, s)$ (resulting situation)
- Successor state axiom: defines how fluents change when an action occurs.

8. Event-Based Representation

- Events are used to represent things that happen at particular times.
- Useful for modeling complex, real-world domains with many interacting events.
- Events can have participants, time, location, and causal relations.

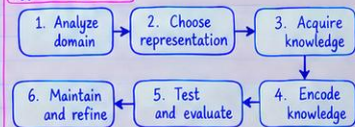
Example: An event representation.

Event:	Earthquake(e1)
Time:	At(e1, t1)
Location:	In(e1, CityX)
Causes:	Causes(e1, BuildingCollapse(e2))
Effects:	Happens(BuildingCollapse(e2), t2)
Participants:	Affects(e2, Buildings)

9. Knowledge Engineering Process

- Knowledge engineering builds a KR system for a real application domain.
- It involves identifying the domain, selecting a representation, encoding knowledge, and testing and refining the system.

Typical Workflow:



10. Applications of Knowledge Representation

- Robotics: represent environment, objects, and actions.
- Natural language understanding: represent meaning of text.
- Planning: represent actions, goals, and world state.
- Expert systems: represent domain knowledge and rules.

- Commonsense reasoning: represent everyday knowledge.
- Vision: represent objects, scenes, and spatial relations.
- Multi-agent systems: represent beliefs, goals, and intentions.
- Scientific domains: represent complex real-world knowledge (e.g., medicine, biology, engineering).

Key Takeaways

- Ontologies organize knowledge into categories.
- Part-whole and causal relations capture important structure.
- Time, actions, and events help represent change.
- Preconditions and effects specify how actions change the world.
- The situation calculus provides a formal framework for dynamic domains.
- Knowledge engineering builds practical KR systems.
- KR has wide-ranging real-world applications.
- Choosing the right representation is crucial for effective reasoning.

Chapter 10: Knowledge Representation

(Revision Page - 5 of 5)

1. Summary of KR Goals

- Represent knowledge about the world in a form that supports inference.
- Enable intelligent behavior by reasoning with facts, objects, relations and general rules.
- Handle incomplete, uncertain or evolving information.
- Support planning, learning and decision making.
- Be expressive enough for real-world domains but computationally tractable.

Key Idea: A good KR tells the agent what exists, how things are related, and how to draw new conclusions.

2. Major Representation Schemes

- Propositional logic - simple facts, true/false.
- First-order logic - objects, relations, quantifiers.
- Semantic networks - graph-like representation.
- Frames - structured objects with slots (attributes).
- Ontologies - formal conceptualization of a domain.
- Default reasoning - typical knowledge and exceptions.
- Inheritance - share and override properties.
- Production rules - IF-THEN rules (actions/inference).
- Probabilistic representations - handle uncertainty (e.g., Bayesian networks).

Different schemes are suited for different problems. The choice depends on the domain and the reasoning needs.

3. How Inference Depends on Representation

- The representation language defines what inferences are possible.
- More expressive languages (e.g., FOL, ontologies) allow richer inferences but are computationally more complex.
- Some representations support efficient inference (e.g., propositional logic, rule-based systems).
- Structure in the representation (e.g., inheritance, ontologies) enables useful generalization.

Key Idea: The form of knowledge determines the kinds of conclusions we can draw and the algorithms we can use.

4. Expressiveness vs. Efficiency Trade-offs

- More expressive representations can model complex real-world knowledge.
- Greater expressiveness often leads to higher computational cost for inference.
- Simpler representations are more efficient but may not capture all relevant knowledge.
- The right balance depends on the application.

Key Idea: Choose a representation that is expressive enough for the task but still allows efficient reasoning.

5. Consistency and Completeness

- **Consistency:** the knowledge base does not contain contradictory information.
 - If inconsistent, anything can be inferred (trivial inferences).
- **Completeness:** the knowledge base contains all relevant facts (with respect to a domain).
 - A complete KB can answer any query about the domain.

In practice, KBs are often incomplete, and we must handle missing or uncertain knowledge using defaults, probabilities, or assumptions.

6. Domain Modeling Tips

- Identify the main objects, their categories (classes), properties and relations.
- Start with a clear ontology of the domain.
- Capture typical knowledge and common-sense rules.
- Include important constraints (e.g., disjoint classes, transitive relations).
- Consider default knowledge and exceptions.
- Keep the representation modular and extensible.
- Use existing ontologies when available (e.g., WordNet).

Good domain models are clear, consistent, well-structured, and focused on the task.

7. Common Mistakes in KR

- Using a representation that is too simple for the problem (loss of important information).
- Making the KB inconsistent (contradictory facts).
- Ignoring exceptions and default knowledge.
- Modeling at the wrong level of abstraction (too detailed or too vague).
- Not considering computational costs.
- Confusing objects, classes, properties and relations.
- Forgetting to handle change and time.

Always check consistency, choose the right level of detail, and consider how inference will work.

8. Exam-Style Quick Questions

1. What is the main goal of knowledge representation?
2. Give two advantages of using ontologies.
3. Explain the difference between a fact and a default:
4. Why is inheritance useful in KR?
5. What are the trade-offs between expressiveness and efficiency?
6. Give one example where default reasoning is needed.
7. How can an inconsistent KB affect inference?
8. Which representation is best suited for uncertain knowledge and why?

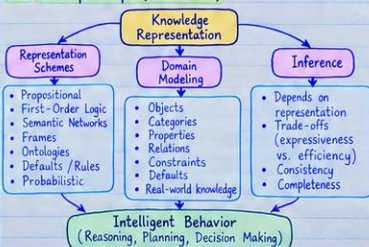
9. Mini Glossary

Term	Meaning
Ontology	Formal specification of a domain: classes, objects, properties and relations.
Fact	A specific piece of knowledge that is true (e.g., Cat(Tom)).
Relation	A connection between objects (e.g., On(Book, Table)).
Frame	Structured object with slots (attributes) and possible values.
Semantic Network	Graph representation with nodes (objects) and links (relations, inheritance).
Default	Typical knowledge that can be overridden by exceptions (e.g., birds usually fly).
Knowledge Base (KB)	A collection of sentences (facts, rules, etc.) representing knowledge about a domain.

10. Chapter 10 Revision Table

Concept	Key Points
Goals of KR	Model the world, support inference, enable intelligent behavior.
Representation Schemes	Propositional, FOL, semantic networks, frames, ontologies, defaults, rules, probabilistic.
Inference & Representation	Language determines what can be inferred and the complexity.
Expressiveness vs. Efficiency	More expressive $\Rightarrow$ higher computational cost. Find the right balance.
Consistency	No contradictions in the KB.
Completeness	All relevant facts (rare in practice).
Domain Modeling	Identify objects, classes, properties, relations, constraints, defaults. Keep it modular.
Common Mistakes	Inconsistency, wrong abstraction level, ignoring exceptions, high complexity, confusion of concepts.

11. Concept Map (Main Ideas)



12. Final Chapter Takeaways

1. KR is about representing information in a form that supports inference.
 2. There are many representation schemes, each with its strengths and limitations.
 3. Representation choice affects what we can infer and how efficiently we can infer it.
 4. More expressiveness usually means higher computational cost.
 5. Consistency and completeness are key properties of a KB.
 6. Good domain modeling is essential for real-world applications.
 7. KR enables reasoning, planning, learning and decision making.
 8. Understanding KR helps us build more capable and intelligent AI systems.
- AIMA (ch.10)

"A good knowledge representation is the foundation for intelligent reasoning and action."